

RQ-VAE Recommender: Semantic-ID tokenization and generative retrieval in PyTorch

Edoardo Botta ¹

¹ Independent Researcher

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Chris Vernon](#) 

Reviewers:

- [@theRizwan](#)
- [@aishukamal](#)
- [@nehak2018](#)

Submitted: 31 August 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

Recommender systems help people navigate large catalogs of products, media, and other items. Conventional retrieval models score catalog items, whereas a generative retriever predicts an item's identifier as a sequence of tokens. RQ-VAE Recommender is a PyTorch research implementation of this approach. It first learns short, hierarchical *semantic IDs* from item descriptions with a residual-quantized variational autoencoder (RQ-VAE). A Transformer then learns to generate the semantic ID of the next item from a user's interaction history. The repository connects data preparation, semantic tokenization, collision handling, constrained generation, and ranking evaluation in a reproducible two-stage workflow.

Statement of need

Semantic IDs replace arbitrary catalog keys with compact codes learned from item features. They can reduce output-vocabulary growth and let related or previously unseen items share statistical structure. The TIGER framework showed that combining residual semantic IDs with sequence-to-sequence prediction can improve retrieval and cold-item generalization ([Rajput et al., 2023](#)). Reproducing that research idea, however, requires coordinated implementations of dataset preprocessing, residual quantization, collision disambiguation, sequential training, corpus-constrained decoding, and top-*K* evaluation.

RQ-VAE Recommender targets researchers and practitioners studying generative recommendation, learned discrete item representations, vector-quantization optimization, and semantic-ID collisions. Its purpose is reproducible training, not inference from pretrained checkpoints. Executable gin configurations and end-to-end PyTorch loops let users locally reproduce data preparation, semantic-ID and retriever training, and ranking evaluation for the supported Amazon and MovieLens workflows; optional experiment tracking and CPU tests support this process. The software runs on a variety of PyTorch-supported hardware, including Apple silicon. As a concrete modest-resource configuration, the author trains the full pipeline locally on an Apple M4 Pro with 48 GB of unified memory rather than data-center hardware.

State of the field

Residual quantization was introduced for hierarchical discrete image representations by Lee et al. ([Lee et al., 2022](#)), while TIGER applied RQ-VAE IDs to generative recommendation ([Rajput et al., 2023](#)). General-purpose libraries such as vector-quantize-pytorch ([Wang, 2020](#)) offer a broad catalog of quantization layers, but do not define the recommendation data, tokenizer, collision, and constrained next-item retrieval workflow implemented here.

Broader generative-recommendation systems have since appeared. GRID combines multiple semantic-ID learners with TIGER-style generation ([Ju et al., 2025](#)), and GenRec is a model

zoo spanning conventional and generative recommenders (Lu, 2025). Those projects serve comparative and large-framework use cases. RQ-VAE Recommender predates them and occupies a narrower niche: a compact end-to-end PyTorch implementation centered on the RQ-VAE-to-retriever interface, with three interchangeable quantization gradient estimators and minimal framework abstraction. Building a standalone implementation made the full pipeline available when the project began and remains useful for focused method development, teaching, and baselines; GenRec explicitly identifies this repository as related prior software.

Software design

The architecture separates semantic-ID learning from next-item generation. In the first stage, an MLP encoder maps item features to a latent vector. A stack of codebooks successively quantizes its residual; codeword indices form a coarse-to-fine ID and the summed codewords are decoded back to the input. Researchers can choose K-means initialization, Euclidean or cosine assignment, and Gumbel-Softmax (Jang et al., 2017), straight-through, or rotation-trick (Fifty et al., 2024) gradients. This separation makes codebook behavior and ID diversity directly inspectable, and lets one tokenizer checkpoint be reused across retrieval experiments, at the cost of not jointly optimizing both stages.

A tokenizer caches IDs for the complete corpus. Because a finite product of codebooks can assign the same tuple to multiple items, it appends a deterministic disambiguation value. The retrieval model embeds the semantic tokens in a shared table and uses a T5-style encoder and autoregressive decoder (Raffel et al., 2020). During generation, it masks partial ID sequences absent from the corpus and keeps the highest-scoring candidates. This enforces catalog-valid outputs while preserving the compact hierarchical vocabulary. Hit rate and normalized discounted cumulative gain provide end-to-end ranking checks.

PyTorch (Paszke et al., 2019) supplies the numerical and accelerator interface, while gin keeps experiment choices outside the training code. The intentionally minimal design exposes ordinary PyTorch modules and configurations for ease of use; separate components let researchers replace one stage without rewriting the pipeline. Training the tokenizer and retriever as separate stages further limits peak resource demand because they do not need to be optimized simultaneously.

Research impact statement

The project has been publicly developed since June 2024 and, as of August 2026, has received more than 840 GitHub stars and 120 forks, with external issues and pull requests documenting use and extension by the community. A pretrained Amazon Beauty tokenizer is distributed through Hugging Face, and version 1.0.1 is preserved in a citable Zenodo archive (Botta, 2026).

More directly, Hu et al. based all public-dataset experiments for their QuaSID research on the open-source RQ-VAE Recommender framework and linked this repository in their implementation details (Hu et al., 2026). Their study used the framework for Amazon Beauty and Toys baselines and for testing new collision-aware semantic-ID objectives. This independent reuse demonstrates that the software functions as an extensible research baseline rather than only as code for a single analysis.

AI usage disclosure

OpenAI Codex using GPT-5 (accessed in August 2026) was used to audit JOSS requirements and assist with initial drafts of packaging metadata, automated tests, repository documentation, and this manuscript. It did not make the core research or software-design decisions. The author

84 reviewed and validated the AI-assisted code changes. The documentation and manuscript were
85 checked against the implementation, cited sources, and automated validation results.

86 Acknowledgements

87 This work received no specific external funding.

88 References

- 89 Botta, E. (2026). *RQ-VAE Recommender* (Version 1.0.1). Zenodo. <https://doi.org/10.5281/zenodo.22076163>
- 91 Fifty, C., Junkins, R. G., Duan, D., Iger, A., Liu, J. W., Amid, E., Thrun, S., & Ré, C. (2024).
92 *Restructuring vector quantization with the rotation trick*. <https://doi.org/10.48550/arXiv.2410.06424>
- 94 Hu, Z., Chen, Y., Pan, Y., Yuan, X., Yin, Y., Wang, D., Xia, B., Luo, Z., Wang, H., Ni, S.,
95 Liang, D., Wang, J., Cai, S., Zhou, T., Ren, F., & Ou, W. (2026). *Stop treating collisions*
96 *equally: Qualification-aware semantic ID learning for recommendation at industrial scale*.
97 <https://doi.org/10.48550/arXiv.2603.00632>
- 98 Jang, E., Gu, S., & Poole, B. (2017). Categorical reparameterization with gumbel-softmax.
99 *International Conference on Learning Representations*. <https://openreview.net/forum?id=rkE3y85ee>
- 101 Ju, C. M., Collins, L., Neves, L., Kumar, B., Wang, L. Y., Zhao, T., & Shah, N. (2025).
102 *Generative recommendation with semantic IDs: A practitioner's handbook*. <https://doi.org/10.48550/arXiv.2507.22224>
- 104 Lee, D., Kim, C., Kim, S., Cho, M., & Han, W.-S. (2022). Autoregressive image generation
105 using residual quantization. *Proceedings of the IEEE/CVF Conference on Computer*
106 *Vision and Pattern Recognition*, 11523–11532. https://openaccess.thecvf.com/content/CVPR2022/html/Lee_Autoregressive_Image_Generation_Using_Residual_Quantization_CVPR_2022_paper.html
- 109 Lu, Q. (2025). *GenRec: A model zoo for generative recommendation*. <https://github.com/phonism/genrec>
- 111 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z.,
112 Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M.,
113 Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch:
114 An imperative style, high-performance deep learning library. *Advances in Neural*
115 *Information Processing Systems*, 32. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- 117 Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu,
118 P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer.
119 *Journal of Machine Learning Research*, 21(140), 1–67. <https://jmlr.org/papers/v21/20-074.html>
- 121 Rajput, S., Mehta, N., Singh, A., Keshavan, R. H., Vu, T., Heldt, L., Hong, L., Tay,
122 Y., Tran, V. Q., Samost, J., Kula, M., Chi, E. H., & Sathiamoorthy, M. (2023).
123 Recommender systems with generative retrieval. *Advances in Neural Information Processing*
124 *Systems*, 36, 10299–10315. https://proceedings.neurips.cc/paper_files/paper/2023/hash/20dcab0f14046a5c6b02b61da9f13229-Abstract-Conference.html
- 126 Wang, P. (2020). *Vector-quantize-pytorch*. <https://github.com/lucidrains/vector-quantize-pytorch>